

Description Of Your Assignment:

Based on the work that you completed in the last assignment, you have now selected a top-level design that you want to use for the creation of the EventHub ticketing application. You have created C4 Context diagrams that show the top level design for the product. A key part of your design was to show how each module will interact with the other modules.

Your task in this assignment is to break down the C4 Context diagrams that you have created further into C4 Container and Component diagrams, based on information made available from Call 1 and 2 initially, and then in Call 3. The goal will be to show how each module will actually be built by a software engineer. The question to be answered is "how will we implement this system?" To answer this question you will have to make concrete technical decisions.

What You Need To Do:

Using the C4 Context diagrams that you have created, you need to use a GenAI tool to create C4 Container and Component diagrams that show the detailed design for the product. Your team made the architecture decisions in the calls. Python and Flask are given. The calls also record the team's position on one application versus several, and on the components. Your design applies those decisions to the containers and components, and it makes the decisions that the calls leave open, such as the database and the interfaces between the components. You will need to identify the data structures and the algorithms to be used in the creation of the EventHub ticketing application. As a part of your design you will specify both APIs and interfaces. Your goal is to produce a design that developers will be able to implement. This model should be optimized for performance, scalability, and maintainability.

Submit your C4 Context diagrams and the details from Call 1 and Call 2 to the GenAI tool in order to create C4 Container and Component diagrams. Having done this, you now need to review the Call 3 transcript and determine how it changes your requirements, modeling, and design as you make new C4 Context, Container and Component diagrams. Compare your old C4 Container and Component diagrams to your new C4 Container and Component diagrams and explain what changed and why.

Eligibility For Grading:

Your team repository must contain the following at the commit on the default branch at the deadline. Both items are required in full. A submission missing any of them receives a grade of 0.

1. Your finalized EARS requirements, updated after Call 3, as markdown files organized by feature:
    - One folder per feature under `features/`, named in lowercase with hyphens, for example `features/pricing/`. The folders follow your own component design. One extra folder, `features/system/`, holds requirements that apply to every feature, such as input validation at the boundary, session expiry, and performance.
    - Each folder holds a `requirements.md` with at least one requirement, one requirement per line, in this form:




```
- **pricing-007** (event-driven): When a buyer
selects a currency, the {EventHub} shall ...
```

- The ID prefix is the folder name. Three-digit numbers start at 001 in each folder. The pattern in parentheses is one of: ubiquitous, event-driven, state-driven, unwanted, optional, complex. Each sentence contains "shall" and opens with the keyword of its pattern: When (event-driven), While (state-driven), If … then (unwanted), Where (optional), two or more of them (complex). The system name is written in braces, as in `{EventHub}`, and comes directly before "shall", after the comma that ends the keyword clause. A ubiquitous requirement has no keyword clause and starts with "`The`" followed by the system name, as in "`The {EventHub} shall ...`", with nothing before "`The`". A complex requirement keeps the EARS order of clauses: Where, While, then When or If.
- IDs are never renamed or reused. A requirement that moves to another folder or is rewritten gets a new ID, followed by an indented line `supersedes: purchase-004`. The old line stays where it was, unchanged otherwise, and ends with `superseded by pricing-009`.

2. Your Structurizr DSL source and rendered diagrams: `workspace.dsl` at the repository root, opening without errors in Structurizr Lite, with the context, container, and component views, and a PNG or SVG export of each view in `diagrams/`. Give each view a key in the DSL and keep the export file name Structurizr Lite produces, which contains that key. Write each view as a multi-line block, with the opening brace at the end of the declaration line and `include *` and the other statements on the lines below, as in the Structurizr documentation. A view written on one line is not counted.

Note: The scripts that we will be using to check your submission will also be shared with you so that you know exactly what we will be checking for.

Example

```
team-repository
├── diagrams
│   ├── component-view.png
│   ├── container-view.png
│   └── context-view.png
├── features
│   ├── checkout
│   │   └── requirements.md
│   ├── pricing
│   │   └── requirements.md
│   └── system
│       └── requirements.md
└── workspace.dsl


Sample content:

team-repository/features/pricing/requirements.md
```




```
- **pricing-001** (event-driven): When a buyer selects a cur
rency, the {EventHub} shall ...
- **pricing-002** (state-driven): While an account email end
s in the USF domain, the {EventHub} shall ...

team-repository/features/system/requirements.md
- **system-001** (ubiquitous): The {EventHub} shall validate ...
```

End Result:

You are required to perform some tasks and submit a zip file (named eventhub-teamname.zip) as part of this assignment to be graded:

1. Updates in your assigned GitHub repository that fulfill both eligibility criteria, i.e., for the EARS formatted requirements, and the Structurizr DSL based C4 diagrams, in full [a submission missing any of them receives a grade of 0].

In Canvas, a zip file that contains two PDF files: (1) an Assignment Report and (2) a Teamwork Reflection.

1. The Assignment Report will contain:
    - The original C4 Container and Component diagrams that were produced with the help of the GenAI tool. [20% of your grade]
    - Your documentation on the changes to requirements, modeling, and design that were made based on new information, **in your own words**. [35% of your grade]
    - The final C4 Container and Component diagrams. [30% of your grade]
2. The Teamwork Reflection is your filled-in copy of the template in the next section. [15% of your grade]

Note that your final grade of the assignment will be dependent on the teamwork reflection and supporting documents. Details in the teamwork reflection section.



CEN 4020 EventHub Assignment Teamwork Reflection

Fill in this form after completing each assignment of the EventHub project and submit it as a PDF called "**Reflection – Gold.pdf**" or "**Relection-Silver.pdf**", with the assignment itself, in the zip file. Screenshots and other supporting documents go inside that PDF. Each Silver and Gold team member submits their own copy. The form is worth 15% of the assignment grade: full credit for a complete form, with every table filled in for every team member, a share of the work that adds up to 100, and specific answers to each question. A member who does not submit a copy receives 0 for that 15%. The purpose is to give the instructor a view of how the work on this assignment was shared among the members of your team, and to give you a moment to think about what worked and what to change before the next assignment. **Your answers will not be shared with anyone else, including your teammate.**



Your Team's Name:

Your Name:

Your Teammates' Names:

Part 1: Roles

The EventHub project is pair programmed, so during this assignment the members of your team took turns as the *"Driver"* (writing the requirements, diagrams, code, or tests) and as the *"Navigator"* (reviewing, questioning, and steering the work). When you work on one screen, the driver adds a `Co-authored-by:` trailer for each navigator to every commit, so the commit history credits everyone who took part. On a scale from 1 (worst) to 10 (best), rate how each team member did in each role during this assignment. Include yourself, one row per team member.

Team Member	Driver	Navigator

Part 2: Collaboration

Your team was expected to collaborate during this assignment to plan the work, divide it, and review what had been done. This required each team member to show up, report on their progress, and contribute. How do you feel that each team member did (scale 1 to 10)?

Team Member	Collaboration

Part 3: Share of the work

Estimate how the work on this assignment was divided. The percentages across all team members should add up to 100.

Team Member	Share of the work (%)



Part 4: Reflection

Answer each question in a few sentences.

1. What did you contribute to this assignment?
2. What did each of your teammates contribute to this assignment?
3. Which AI tools did you use on this assignment, for which pieces of the work, and how did you check the output before it went into the submission?
4. What went well in how your team worked together?
5. What will you change for the next assignment?
6. Is there anything the instructor or the TA should know about how this assignment went?

Depending on the concerns raised and the documents supporting them (for example, screenshots, or a commit history with co-author trailers showing who took part in each piece of work), the grade per assignment may get affected for one or more team members. All copies from the same team will be compared and considered.



Call 3: Design, Containers & Components

Adaeze [Director]: Design call. The architecture call gave us the outside view; today is the inside. Priya [Lead Dev], take us through it.

Priya [Lead Dev]: Two levels today. First the containers, the separately runnable pieces: applications, databases, stores. Then the components inside the main application, the clusters of responsibility. I will lay out the options, and the team building it makes the decisions.

Marcus [Product Manager]: Containers first, then.

Priya [Lead Dev]: The provisional decision from the architecture call was a single web application. If that holds, the container view is small: the Flask application itself, and a database. The database choice is the first real fork: SQLite is a single file that needs no setup and works well for development and modest traffic, but it handles writes one at a time, in a queue. If two hundred buyers press “buy” in the same second…

Kenji [Security]: They queue, briefly. For our launch traffic, acceptable.

Priya [Lead Dev]: Likely, yes. But “likely acceptable” should be written in a decision record next to the load numbers rather than said in a meeting. Next: where do exchange rates live? The architecture call left “someone edits something” hanging.

Marcus [Product Manager]: So, update on that. I talked to finance this week. Rates will change more often than we thought. Monthly, maybe weekly if a currency moves a lot. Editing a file and re-releasing the application every time is not workable. They want a screen where they log in and change the number.

Priya [Lead Dev]: That’s a genuinely new fact, and everyone should hear what it changes. A screen where someone edits rates means an *administrator*: a second user role, with permissions a buyer must never have. The architecture call concluded one human actor. That conclusion is now wrong. The top-level context picture has to be redrawn: two actors, and a buyer must never be able to do what an administrator does.

Thuy [QA]: While the admin area is being changed, I have a request. I asked for the audit log in Call 1, but we never said how anyone *reads* it. If an admin area is being added, the log viewer belongs inside it. Investigating a dispute should not require someone opening database files manually.

Priya [Lead Dev]: Reasonable, and it compounds the point: the admin area is now rates *and* audit access. Anyone updating diagrams from the architecture call, take note: the context level changes, the container level gains admin-facing routes, and the component level gains an admin component. That rework is real work; better now than after code exists.

Kenji [Security]: And that raises a question nobody has answered: who *is* an administrator? How does an account become one? Is it a flag, a role, a separate login?

Marcus [Product Manager]: Finance staff, I assume. Or us. We’ll figure out the mechanism.

Kenji [Security]: Hm.

Adaeze [Director]: Components next. The inside of the application.



Priya [Lead Dev]: Listening to everything so far, responsibilities cluster naturally: one component handles identity: registration, login, sessions. One serves the catalog: browsing, search, event pages. One executes purchases. One computes prices, and I mean exactly *one*: base price, quantity, discount eligibility, currency conversion, all meeting in a single place. One records what happened, the audit trail. And now, one for administration. The exact decomposition is the design work; those boundaries are where I would start.

Thuy [QA]: One question on the catalog before we go further. When a search by artist or venue matches nothing, what does the page show? My browser tests need an expected result for that case, and nobody has written one.

Priya [Lead Dev]: Three common treatments. An empty list with no explanation, a message that says no events match the search and keeps the search box on the page, or that same message followed by the full event list so the user can browse instead.

Marcus [Product Manager]: The message followed by the full list. A student who typed a name we don't have should see what we do have, and an empty page looks like a broken site.

Kenji [Security]: One condition. If the message repeats what the user typed, the page must treat that text as plain text, never as markup.

Adaeze [Director]: Decided. No match shows a message that no events match the search, followed by the full event list, and the search text is shown as plain text only.

Thuy [QA]: Back to pricing. Why so insistent on it being one place?

Priya [Lead Dev]: Because of the unresolved math. Whether the discount applies before or after conversion is still an open question from Call 1. If pricing logic lives in one component, the answer arrives and it's a one-place change. If it's smeared across five templates, the answer arrives and you fix four of the five, and you find the fifth in production.

Kenji [Security]: On money, while we're here. This is a constraint rather than a solution: computers do surprising things to decimal money if you let them, and currencies disagree about decimal places: yen has none. If the build doesn't make a deliberate choice about how money is represented, the machine will make it silently, and it will surface in exactly one currency, months from now.

Priya [Lead Dev]: Strongly seconded. Write the choice down in the decision record.

Marcus [Product Manager]: Different topic. Checkout flow, since we're designing it. One thing I want: once a buyer starts checkout, lock them in. Disable the back button, stop them closing the tab, until they either finish or press cancel. Half-finished checkouts are data garbage.

Priya [Lead Dev]: What problem are we solving? Abandoned checkouts polluting data, or something else?

Marcus [Product Manager]: Abandoned checkouts, mostly. I want clean funnels.

Priya [Lead Dev]: Then let's note "handle abandoned checkouts cleanly" as the need and let the build decide the treatment.



Marcus [Product Manager]: As long as the lock-in is on the list.

Kenji [Security]: While we're on checkout, one thing I left open in the requirements call. I said checkout and order history require login. I never said what the application does when a user who is not signed in opens one of those pages.

Priya [Lead Dev]: Two common treatments. Show an error page that says login is required, or redirect the user to the login page and, after a successful login, send the user back to the page they asked for.

Marcus [Product Manager]: Redirect them to login and bring them back. A student who was one click from buying shouldn't have to find the event again, and an error page loses the sale.

Kenji [Security]: Agreed, with two constraints. Nothing from those pages is shown before login, and no order or checkout record is created for a user who is not signed in. That applies to every checkout and order history page, including a link pasted straight into the address bar.

Adaeze [Director]: Decided, then. Redirect to login, return to the requested page, and nothing shown or recorded before login.

Kenji [Security]: Session expiry, related to checkout. I still think thirty minutes idle.

Marcus [Product Manager]: And I still think students will scream. They are already annoyed about the two-factor authentication that logs them out of every single USF service now and then. Every ticketing site keeps you signed in for weeks.

Kenji [Security]: Shared library computers, Marcus [Product Manager].

Marcus [Product Manager]: Their own phones, Kenji [Security].

Priya [Lead Dev]: Moving to interactions, the arrows between components. The one that deserves the most design attention: purchase touches identity, catalog, pricing, inventory, and audit, in that order, and the availability check plus the count decrement must be one indivisible step. That's the requirement Marcus [Product Manager] wrote after the architecture call. Whoever draws the component diagram: spend most of your diagram effort on that path, because it's where the mistakes will be.

Thuy [QA]: And the audit component: I want it *in* the purchase path, not beside it. If each route remembers to log, someone forgets. If logging is part of the path itself, it can't be skipped.

Priya [Lead Dev]: Agreed. One component owns logging instead of every route doing its own. Also worth drawing: the admin area is only reachable by administrators, and *nothing* in the buyer-facing flow calls into it.

Adaeze [Director]: What should exist on paper at the end of this phase?

Priya [Lead Dev]: A container view: application, database, and whatever the rates decision produces. A component view of the application interior, showing the purchase path and which components only administrators can reach. A revised context view, two actors now. And decision records for the forks: database choice, money representation, rates storage. A well-argued decision record matters more than a pretty diagram.

Adaeze [Director]: Then we're adjourned. Implementation call in three weeks.